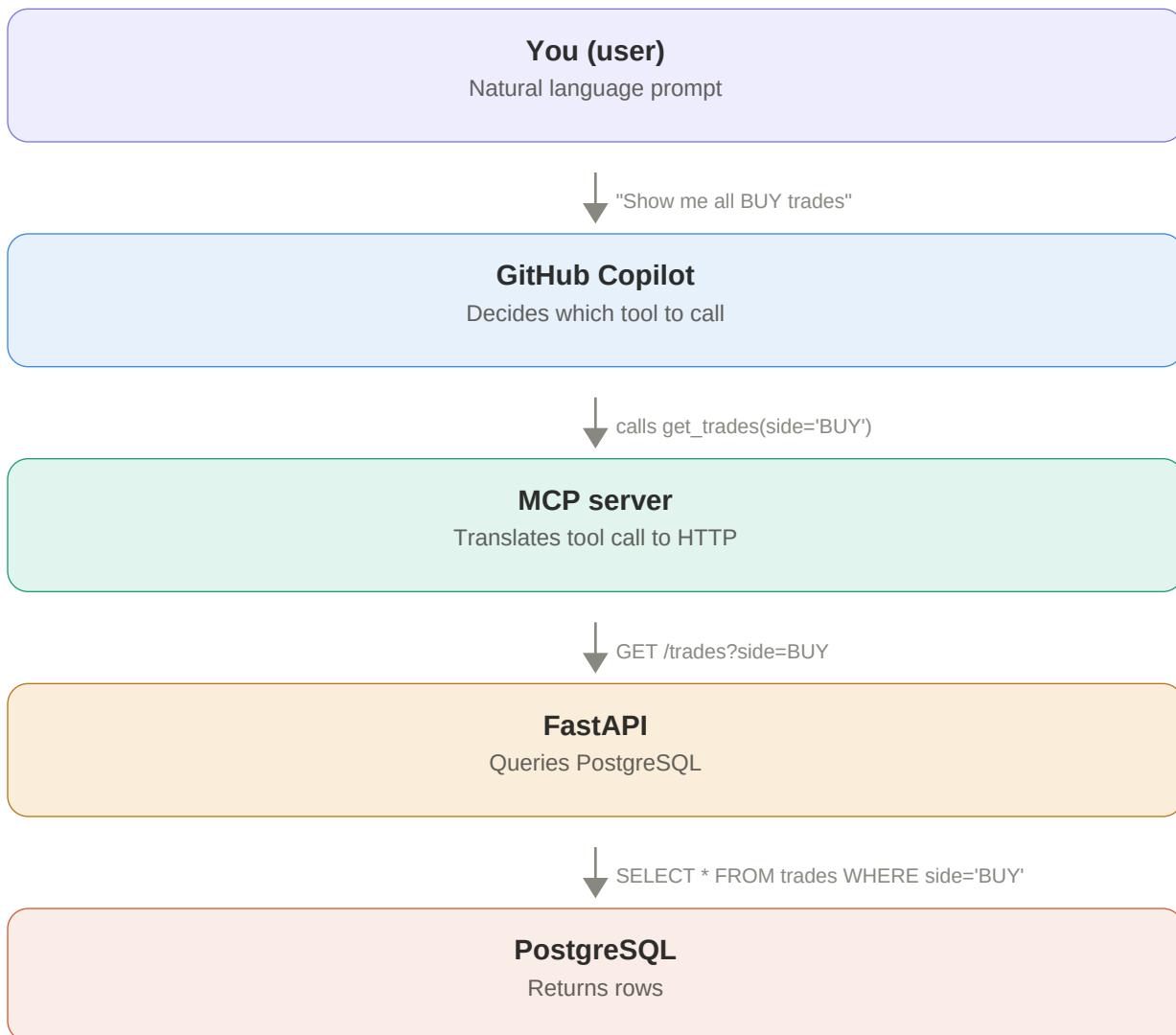


Trades Agent — Architecture Explainer

This document explains how GitHub Copilot, the MCP server, FastAPI, and PostgreSQL work together to form a natural-language trades agent. Each section covers one layer of the stack — what it does, why it exists, and how data flows through it.

1. End-to-end agent flow

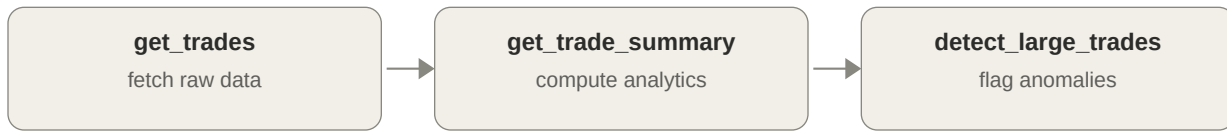
When you type a prompt in Copilot's agent mode, it travels through four layers before a response reaches you. Each layer has a single responsibility.



The dashed return path carries the result back up: PostgreSQL rows → FastAPI JSON → MCP tool result → Copilot reasoning → your answer.

The agentic loop

Copilot can call multiple tools in a single turn. After each tool result it reasons again: 'Do I need more data?' This loop continues until it has everything needed for a complete answer.



Example multi-tool chain for: 'Analyze my portfolio and flag any risks'

2. What GitHub Copilot does with your prompt

Copilot in agent mode is a reasoning loop powered by a large language model (GPT-4o). It does not execute code itself — it decides what to do and delegates to tools.



Final response

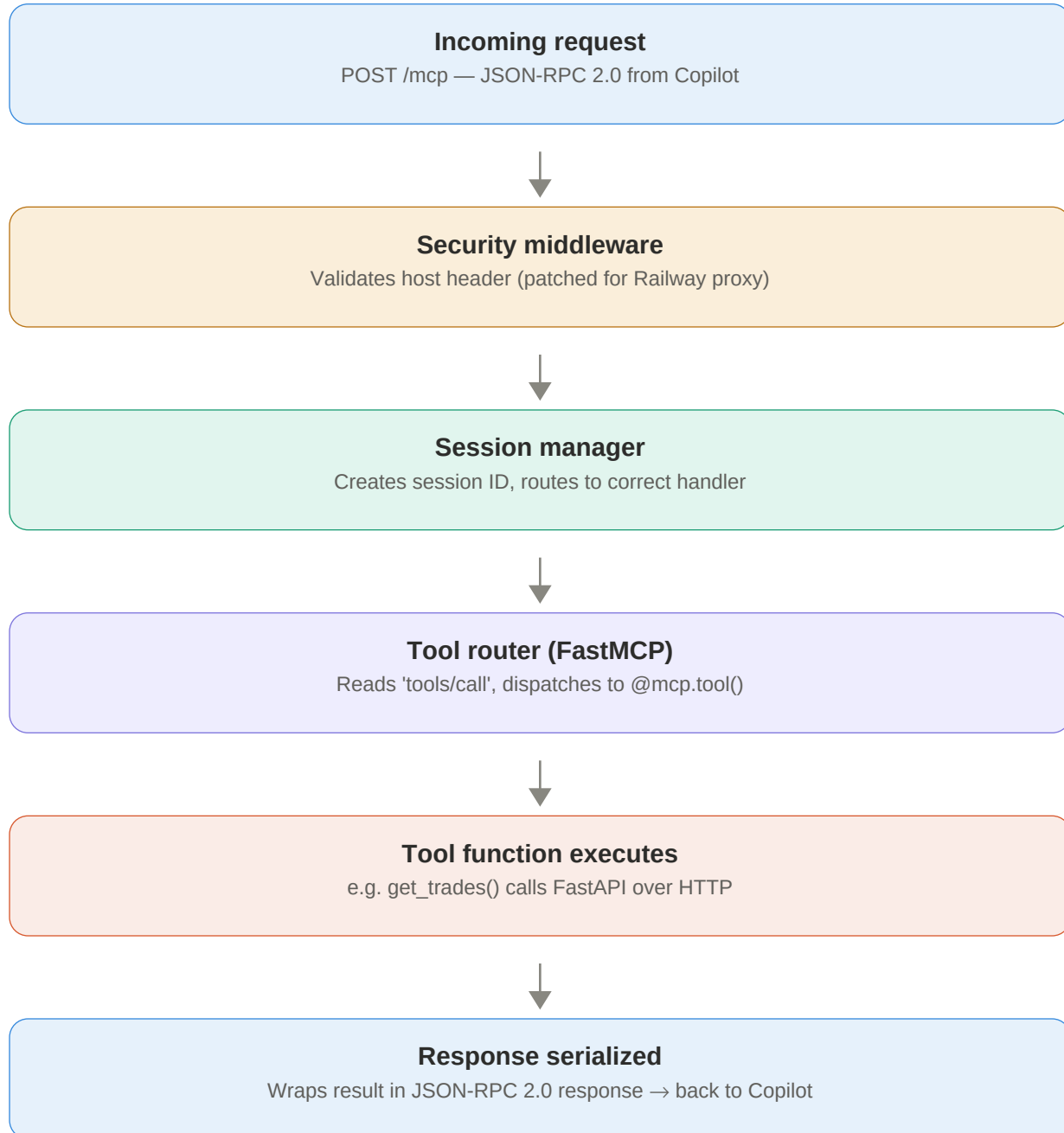
Synthesized answer from all tool results

Key concepts

Concept	Explanation
Tool schema	Each <code>@mcp.tool()</code> function exposes its name, description, and input types. Copilot reads these to decide which function to call.
JSON-RPC 2.0	The wire protocol between Copilot and the MCP server. Every tool call is a POST with method: 'tools/call'.
LLM reasoning	The LLM reads: prompt + tool list + history → outputs a tool call decision. It never runs code itself.
Agentic loop	After each tool result the LLM reasons again. <code>stop_reason='end_turn'</code> ends the loop.

3. What the MCP server does

The MCP server is a translator. On one side it speaks the MCP protocol to Copilot. On the other it makes plain HTTP calls to your FastAPI. Your `@mcp.tool()` functions are the bridge between both worlds.



Registered MCP tools

Tool	Description
get_trades	List trades with optional filters: symbol, side, limit, offset

get_trade_by_id	Fetch a single trade by its database ID
create_trade	Insert a new trade: symbol, side, quantity, price
get_trade_summary	BUY/SELL breakdown, top symbols, value by symbol
get_largest_trades	Top N trades by total value (quantity x price)
get_trades_by_symbol_analysis	Avg price, net position, long/short status per symbol
detect_large_trades	Flag trades above a configurable value threshold
get_recent_trades	Last N trades ordered by timestamp

Railway deployment fix

Railway routes traffic through a reverse proxy that changes the Host header. The MCP SDK's `transport_security.py` rejects unexpected hosts with a 421 error. The fix: monkey-patch `StreamableHTTPServerTransport` to replace the security class with a permissive `AllowAllSecurity` that always returns `None` (no error).

```
class AllowAllSecurity:
    async def validate_request(self, request, is_post=False):
        return None # allow all hosts

StreamableHTTPServerTransport.__init__ = patched_init
```

4. Full stack summary

Layer	Technology	Responsibility	Deployed on
Agent UI	GitHub Copilot	Natural language → tool calls	VS Code
MCP server	Python + FastMCP	Protocol bridge + analytics tools	Railway
API	FastAPI + asyncpg	REST endpoints + DB queries	Railway
Database	PostgreSQL	Persistent trade storage	Railway (managed)

Example prompts you can use in Copilot agent mode:

- Get all trades
- Show me all SELL trades
- Analyze my trades and give me a full summary
- What's my net position on AAPL? Am I long or short?
- Flag any trades over \$5,000 in value
- What are my 5 largest trades?
- Show me the last 10 trades and identify any patterns
- Create a BUY trade for NVDA, 10 shares at \$900